◆ What is Procedural Programming?

Procedural Programming is a programming style where a program is written as a **sequence of step-by-step instructions (procedures or functions)**.

It focuses on:

- Functions (procedures)
- Step-by-step execution
- Data is passed between functions

◆ What is Object-Oriented Programming (OOP)?

Object-Oriented Programming (OOP) is a programming paradigm where we design software using **objects** instead of step-by-step procedures.

It focuses on:

- Objects (real-world entities)
- Data (state)
- Behavior (methods)

☞ In OOP, data and functions are combined inside objects.

◆ Difference between Procedural and OOP Programming

Procedural Programming	Object-Oriented Programming
Focus on functions	Focus on objects
Data and functions are separate	Data and functions are combined
Hard to manage large programs	Easier to manage large programs
Code is less reusable	Code is reusable
Example: function(data)	Example: object.method()

☞ Procedural = “What steps to do”
☞ OOP = “Who does what”

◆ Why do we need OOP?

We need OOP because in large programs:

- Data and functions become difficult to manage in procedural programming
- Code becomes messy and hard to maintain
- It is difficult to track which function modifies which data

OOP solves these problems by:

- Combining data and behavior in one unit (object)
- Making code more organized and reusable
- Making large systems easier to manage

◆ What is a Class?

A Class is a **blueprint of an object** that are used to create an objects. It does not exist physically in memory as data

It defines:

- What data an object will have (attributes)
- What actions an object can perform (methods)

Example: Blueprint of a house

> Let's define a simple `Dog` class with one attribute (`name`) and one method (`bark`).
>
> **C++**
>
> ```cpp
> class Dog {
> public:
>     string name;
>
>     void bark() {
>         cout << "Woof!";
>     }
> }; // Semicolon required!
> ```
>
> **Java**
>
> ```java
> class Dog {
>     public String name;
>
>     public void bark() {
>         System.out.println
>             ("Woof!");
>     }
> }
> ```
>
> **Python**
>
> ```python
> class Dog:
>     # State defined in
>         constructor
>     def __init__(self, n):
>         self.name = n
>
>     def bark(self):
>         print("Woof!")
> ```

How do we bring the blueprint to life? This is called **Instantiation**.

C++

```
// Created on the Stack
Dog myDog;
myDog.name = "Rex";
myDog.bark();
```

Java

```
// Created on the Heap
Dog myDog = new Dog();
myDog.name = "Rex";
myDog.bark();
```

Python

```
# Created on the Heap
my_dog = Dog("Rex")

my_dog.bark()
```

Notice the Dot Operator (.). We use it to reach inside the object to access its state or behavior.

◆ What is an Object?

An Object is a **real instance of a class**.

- It is created from a class
- It exists in memory (RAM)
- We can create multiple objects from one class

Example: Actual houses built from the same blueprint

◆ Difference between Class and Object

Class	Object
Blueprint or design	Real instance
Does not occupy memory	Occupies memory
Logical concept	Physical existence in program
Defines structure	Holds actual data

1.3 State and Behavior (Clean & Exam-Ready)

An **Object** in Object-Oriented Programming is a software entity that combines **two important parts: State and Behavior**.

1. State (Attributes / Data)

State means **what the object is or what it knows**.

- It represents the **data or properties** of an object
- It is stored using **variables (attributes)** inside the object

☞ Example (Dog object):

- name = Bruno
- breed = Husky
- age = 3
- color = Brown

✓ So, state describes the **current information** of an object.

⚙ 2. Behavior (Methods / Logic)

Behavior means **what the object can do**.

- It represents the **actions or functions** of an object
- It is defined using **methods (functions inside the class)**

☞ Example (Dog object):

- bark()
- eat()
- sleep()

✓ So, behavior describes the **actions or functionalities** of an object.

◆ What is a Custom Data Type?

A **Custom Data Type** is a user-defined data type created by programmers when built-in data types (like int, float, char, boolean) are not enough to represent real-world objects.

? Simple Explanation:

Computers already provide basic data types:

- `int` → numbers
- `float` → decimal numbers
- `boolean` → true/false

These are called **primitive data types**.

But in real life, we often deal with complex entities.

🏗️ Why do we need Custom Data Types?

For example, consider a **Student**.

A student has:

- Name (string)
- ID (int)
- GPA (float)

👉 These values belong to one entity but are of different types.

So, instead of managing them separately, we create a new type called:

👉 `Student` (Custom Data Type)

💡 Class = Custom Data Type

When we create a **class**, we are actually defining a **new custom data type**.

Example:

```
class Student {
public:
    string name;
    int id;
    float gpa;
};
```

👉 Here, `Student` is a custom data type created by the programmer.

Stacks:

A stack is a list of elements in which an element may be inserted or deleted only at one end, called the top of the stack.

Two basic operations associated with stacks:

"**Push**" is the term used to insert an element into a stack.
"**Pop**" is the term used to

Memory in a computer program is mainly divided into two parts: **Stack** and **Heap**. These two areas are used to store different types of data during program execution.

---

1. Stack Memory

The **Stack** is a small, fast, and structured area of memory used for storing **local variables** and **function calls**.

- It follows the **LIFO (Last In, First Out)** principle.
- Each time a function is called, a new block (called a stack frame) is created.
- When the function finishes execution, its allocated memory is automatically removed.

Characteristics:

- Very fast access ⚡
- Limited in size
- Automatically managed (no need for manual deallocation)
- Stores primitive data types and local variables

Example:

```
void MyFunction() {
    int x = 10;
    int y = 20;
}
```

Here, `x` and `y` are stored in the Stack.

2. Heap Memory

The **Heap** is a large and unstructured area of memory used for **dynamic memory allocation**.

- Objects and data created at runtime are stored in the Heap.
- Memory is allocated manually (or by using `new`) and managed by the **Garbage Collector** in languages like C#.

Characteristics:

- Slower than Stack 🐢
- Much larger in size
- Used for storing objects and complex data
- Requires memory management (automatic in .NET via Garbage Collector)

Example:

```
Student s = new Student();
```

Here, the object is stored in the Heap, while the reference variable `s` is stored in the Stack.

---

3. Difference Between Stack and Heap

Feature	Stack	Heap
Speed	Very fast	Slower
Size	Small	Large
Memory Management	Automatic	Manual / Garbage Collected
Data Stored	Local variables, primitives	Objects, dynamic data
Structure	Organized (LIFO)	Unstructured

Conclusion

In summary, the **Stack** is used for temporary data like function calls and local variables, while the **Heap** is used for dynamic memory allocation such as objects. Understanding the difference between them is essential for efficient memory management and programming.

When you write Dog d1 = new Dog(); in Java (or d1 = Dog() in Python), what is d1?

In conclusion, `d1` is a **reference variable stored in Stack memory**, which holds the **address of a Dog object located in Heap memory**. The actual object data is stored in the Heap and all operations on the object are performed through the reference variable.

In the statement `Dog d1 = new Dog();`, **`d1` is a reference variable (or pointer)**, not the actual object.

- The **actual `Dog` object is created in the Heap memory**.
- The **reference variable `d1` is stored in the Stack memory**.

`d1` stores the **memory address (reference)** of the object created in the Heap.

Step 1: Object Creation

```
new Dog();
```

- A new `Dog` object is created in the **Heap memory**.
- Memory is allocated for its data (e.g., name, age).

Step 2: Reference Assignment

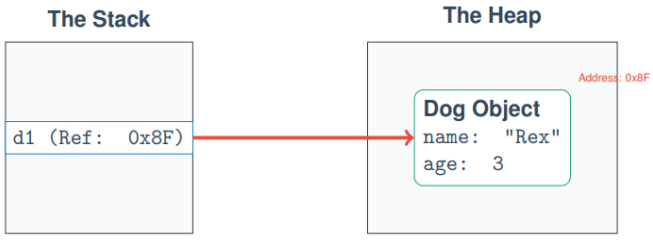
```
Dog d1;
```

- A variable `d1` is created in the **Stack memory**.
- This variable is capable of storing a reference (address).

Step 3: Linking Stack and Heap

```
Dog d1 = new Dog();
```

- The **address of the Dog object** (e.g., `0x8F`) is stored inside `d1`.
- So, `d1` now points to the object in the Heap.

> **4.3 Visualizing Object Memory**
>
> 
>
>
> *The variable just holds the memory address of the object.*

Left Side (Stack)

- There is a variable:
  `d1 (Ref: 0x8F)`
- This means `d1` is storing the address `0x8F`

Right Side (Heap)

- At address `0x8F`, there is a **Dog object**
- It contains:
  - name = "Rex"
  - age = 3

What happens if we copy a reference?

Here is a **clear, well-organized exam-ready answer** for *"The Danger of Aliasing"* 👇

The Danger of Aliasing

✓ Definition

Aliasing occurs when **two or more reference variables point to the same object in memory (Heap)**.
In such cases, any change made through one reference affects the same object seen by the other reference.

Given Code

```
Dog d1 = new Dog();
d1.name = "Rex";

Dog d2 = d1;   // Reference copy (Aliasing)

d2.name = "Max";

System.out.println(d1.name);
```

Step-by-Step Explanation

🔹 Step 1: Object Creation

```
Dog d1 = new Dog();
```

- A `Dog` object is created in the **Heap**.
- `d1` (in Stack) stores the reference (address) of that object.

🔹 Step 2: Assign Value

```
d1.name = "Rex";
```

- The object's `name` property becomes **"Rex"**.

🔹 Step 3: Reference Copy (Aliasing)

```
Dog d2 = d1;
```

- No new object is created ✗
- **Only the reference is copied ✓**
- Now, both `d1` and `d2` point to the **same object in Heap**

🔹 Step 4: Modify Using Second Reference

```
d2.name = "Max";
```

- The same object is updated
- So the name becomes **"Max"**

🔹 Step 5: Output

```
System.out.println(d1.name);
```

☞ Output will be: **Max**

Why Output is “Max”?

Because:

- `d1` and `d2` are pointing to the **same object**
- Changing the object through `d2` also affects what `d1` sees

Important Concept

☞ **No new object is created during `Dog d2 = d1;`**
☞ Only the **reference (memory address)** is copied
☞ This leads to **aliasing**

Real-Life Analogy

- `d1` = Remote Control 1 🖥
- `d2` = Remote Control 2 🖥
- Object = TV

➞ Both remotes control the **same TV**
➞ Changing channel with `d2` also affects what `d1` sees

C++ is Different (Pass by Value by Default)

✓ Main Concept

In **C++**, objects are **stored directly in Stack memory by default**, and **copying a variable creates a complete duplicate (clone) of the object**, not just a reference.

1. Object Creation in C++

```
Dog d1;
```

- A `Dog` object is created **directly in the Stack memory**
- No Heap allocation is involved here

2. Object Copy (Pass by Value)

```
Dog d2 = d1;
```

- A **new separate object** `d2` is created

- C++ performs a **deep copy (physical clone)** of `d1`
- Now, `d1` and `d2` are **completely independent objects**

☞ Changing `d2` will **NOT affect** `d1`

What is Encapsulation?

Encapsulation **is one of the fundamental concepts of Object-Oriented Programming (OOP). It means** bundling data (variables) and methods (functions) together into a single unit called a class**, and** restricting direct access to some parts of that data**.**

⋆ Why do we use Encapsulation?

We use encapsulation to protect the **Object Invariant**.

◆ What is an Invariant?

An **invariant** is a rule or condition that must always remain true for an object to be valid.

If this rule breaks, the object becomes invalid or incorrect.

Example (Bank Account)

Think about a **Bank Account**:

✕ Without Encapsulation (Bad Design)

If balance is public, anyone can do this:

- balance = -999999 ✕ (invalid)
- balance = "Potato" ✕ (meaningless)

This breaks the rules of a real bank account.

Access Modifiers:

Access Modifiers are keywords used in Object-Oriented Programming to control the visibility (access level) of class members such as variables and methods.

☞ In simple words:
They decide who can see or use the data inside a class.

Why do we use Access Modifiers?

We use access modifiers to enforce **Encapsulation**.

☞ Encapsulation ensures that data is protected and cannot be accessed directly from outside the class.

Access modifiers help by:

- hiding sensitive data
- controlling how data is accessed or modified
- improving security and data integrity

1. Public (`public`)

Meaning:

- Accessible from **anywhere**
- Outside the class can read and modify it

Usage:

- Usually used for **methods (behaviors)**

Example idea:

If something is public, anyone can use it freely.

☞ Like: ATM machine buttons (withdraw, deposit)

2. Private (`private`)

Meaning:

- Accessible **only inside the same class**
- No outside access allowed

Usage:

- Usually used for **variables (state/data)**

Example idea:

If something is private, only the owner can control it.

☞ Like: ATM internal account balance (you cannot directly change it)

Getters and Setters:

When class variables are declared as **private**, they cannot be accessed directly from outside the class.
So, we use special **public methods** called **Getters and Setters** to safely access and modify those private variables.

```
class Student {
    private String name;  // private variable

    // Setter method (value set করার জন্য)
    public void setName(String name) {
        this.name = name;
    }

    // Getter method (value পাওয়ার জন্য)
    public String getName() {
        return name;
    }
}



public class Main {
    public static void main(String[] args) {
        Student s1 = new Student();
        s1.setName("Rahim");
        System.out.println(s1.getName());

    }

}
```

🔐 Why do we need Getters and Setters?

Because of **Encapsulation**:

- Data is kept **private (hidden)**
- But we still need controlled access
- So we use **public methods as "gatekeepers"**

📌 1. Getter Method (Read Access)

Meaning:

A **getter** is a method used to **read or get the value** of a private variable.

Features:

- Only returns value
- Does NOT modify data

☞ Think: "I want to see the value"

📌 2. Setter Method (Write Access)

Meaning:

A **setter** is a method used to **change or update the value** of a private variable.

Features:

- Can include rules/validation
- Controls how data is updated

👉 Think: "I want to change the value safely"

💻 Example (Bank Account)

```
class BankAccount {
    private double balance = 0; // Hidden data

    // Getter → Read balance
    public double getBalance() {
        return balance;
    }

    // Setter → Controlled update (deposit)
    public void deposit(double amount) {
        if (amount > 0) {
            balance += amount;
        }
    }
}
```

Why not just make it public?

If we make all data **public**, it becomes very easy to access and modify it directly.
But this breaks **Encapsulation** and can lead to invalid or unsafe data.

So instead, we use **private data + public methods** to control access.

🎰 Vending Machine Analogy

This is the best way to understand it:

❌ Public Data (Bad Design)

Imagine a vending machine where everything is open:

- You can directly take chips without paying
- Anyone can open the machine and change items

- No rules are enforced

👉 Result:

- No security
- No control
- System becomes useless

🔐 Private Data + Methods (Good Design)

Now imagine a proper vending machine:

- The items are inside a **locked glass box**
- You cannot directly touch them
- You must use buttons and coin slot

How it works:

1. You insert money
2. You press a button
3. Machine checks payment
4. Machine gives item

👉 The machine enforces all rules automatically

💡 Connection to Programming

Public variables:

- Anyone can change data directly
- No validation
- Risk of invalid values

Private variables + methods:

- Data is hidden
- Access only through methods
- Rules are always checked

Encapsulation in Python (Easy Version)

Step 1: What is Encapsulation?

Encapsulation = **hiding internal data** of a class so that it cannot be accessed directly from outside.

- Helps protect data.
- Only certain methods can access or modify it.

Think of it like a **bank vault**:

- You can **deposit** or **withdraw** money through a controlled system.
- You **cannot directly touch the money** inside.

Step 2: How Python Does Encapsulation

- Python does **not have strict `private` or `public` keywords** like Java or C++.
- Instead, Python uses a **naming convention** to indicate private variables.

Rule:

- Prefix a variable with __ (double underscore) → makes it "hidden"

Example:

```
class BankAccount:
    def __init__(self):
        self.__balance = 0

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount
```

Step 3: Accessing Hidden Variables

- You **should not** access `__balance` directly outside the class.
- Instead, you **use methods** to interact with it.

```
account = BankAccount()
account.deposit(100)   # Correct way
print(account.__balance)  # ✕ This will give an error
```

6.1 The Birth of an Object (Python Version)

When we write:

```
my_dog = Dog()
```

Two things happen:

1. **Memory allocation:** Python asks the OS for memory to store the object.
2. **Constructor call:** Python automatically runs a special method called the **constructor**.

Step 2: What is a Constructor?

Definition:

A constructor is a special method that is automatically called **when an object is created**.

Purpose:

- Initialize the object's **state** (set starting values for variables)
- Ensure the object starts in a **valid state**

Think of it as **giving a baby a name and some clothes right after birth**—the object is ready to use.

6.2 Constructor Syntax

In C++ / Java

- **Name:** Same as the class name
- **Return type:** None (not even `void`)
- **Example (Java):**

```
class Student {
    private String name;

    // Parameterized Constructor
    public Student(String startingName) {
        name = startingName;
        System.out.println("A student is born!");
    }
}

// Usage:
Student s1 = new Student("Alice");  // Must provide a name
```

In Python

- **Name:** Always `__init__`
- **No need to declare variables before using them**; just use `self.variable = value`

Example:

```
class Student:
    # Constructor
    def __init__(self, starting_name):
        self.name = starting_name
        print("A student is born!")

# Usage
s1 = Student("Alice")
```

Notes:

1. `self` refers to the **current object**
2. Assigning `self.name = starting_name` creates the attribute dynamically

Default Constructors

Step 1: What is a Default Constructor?

- A **default constructor** is a **constructor with no arguments**.
- If you **don't write any constructor**, the **compiler automatically provides one** (C++/Java).
- Its job: initialize variables to default "zero values":
    - `0` for numbers
    - `null` for objects
    - `false` for booleans

Step 2: Example in Java/C++

Without writing a constructor:

```
class Student {
    private String name;
    private int age;
}

// Usage:
Student s1 = new Student();  // Default constructor is automatically used
```

- Here, `name = null` and `age = 0` by default.

Step 3: Custom Constructor Effect

- If you write **any constructor yourself**, e.g.:

```
public Student(String name) {
    this.name = name;
}
```

- **Then the compiler removes the default constructor**

When it works

- Works **only if there is a constructor that takes no arguments**.
- If you **didn't write any constructor at all**, the compiler automatically provides a **default constructor** (no-arg). ✓ Works

Example:

```
class Student {
    private String name;
}

Student s2 = new Student();  // ✓ Works, compiler provides default
constructor
```

When it gives ERROR

- You **wrote a custom constructor with arguments** but **did not write a no-argument constructor**.

Example:

```
class Student {
    private String name;

    // Custom constructor with argument
    public Student(String name) {
        this.name = name;
    }
}

// Now:
Student s2 = new Student();  // ✗ ERROR: no default constructor available
```

Reason:

- As soon as you write **any constructor**, the compiler **does not create the default no-argument constructor**.
- So `new Student()` cannot find a matching constructor → **compile-time error**.

The Death of an Object (Destructors)

Step 1: What happens when we're done with an object?

- An object **takes memory** when created.
- Once we **don't need it anymore**, its memory should be **freed** to avoid waste.
- This is called **destruction** of the object.

Step 2: How different languages handle it

✅ ◆ English Version (Short Exam Answer)

Garbage Collection is an automatic memory management process that removes unused or unreachable objects from the heap memory.

In languages like **Java and Python**, objects are stored in heap memory and accessed through references. When an object no longer has any reference, it becomes unreachable, and the garbage collector automatically deletes it to free memory.

In contrast, **C++** does not support automatic garbage collection. The programmer must manually deallocate memory using `delete`, otherwise memory leaks may occur.

✅ ◆ Bangla Version (Short Exam Answer)

Garbage Collection হলো একটি automatic memory management process, যেখানে unused বা unreachable object গুলোকে heap memory থেকে সরিয়ে ফেলা হয়।

Java এবং Python এ object গুলো heap memory তে থাকে এবং reference এর মাধ্যমে access করা হয়। যখন কোনো object এর আর কোনো reference থাকে না, তখন সেটি unreachable হয়ে যায় এবং garbage collector সেটিকে automatically delete করে memory free করে।

অন্যদিকে, C++ এ automatic garbage collection নেই। Programmer কে manually `delete` ব্যবহার করে memory free করতে হয়, না হলে memory leak হতে পারে।

C++ (Manual Memory Management)

- If you created an object **on the heap**:

```
Dog* myDog = new Dog();
```

- You **must delete it manually**:

```
delete myDog;   // Calls destructor ~Dog() automatically
```

- **Destructor (`~Dog()`)**: a special method that cleans up resources (memory, files, etc.) before the object is removed.

Java / Python (Automatic Garbage Collection)

- You **do NOT delete objects manually**.
- The **interpreter/VM tracks references**:
  - If **no variable points** to the object, it's unreachable.
- **Garbage collector** automatically deletes it and frees memory.

Example in Python:

```
class Person:
    def __init__(self, name):
        self.__name = name   # private variable

    def set_name(self, name):
        if isinstance(name, str) and name.strip() != "":
            self.__name = name
        else:
            print("Invalid name!")

    def get_name(self):
        return self.__name


    # When the function ends, myDog has no references
    # Garbage Collector removes the object automatically
```

- No need to write destructors like in C++

Step 3: Key Points for Exam

1. **C++:** Manual deletion (`delete`) → destructor called
2. **Java/Python:** Automatic deletion → garbage collector frees memory
3. **You rarely write destructors in Java/Python**

How Methods Know Which Data to Use

Step 1: The Situation

You have:

```
Dog d1 = new Dog("Rex");
Dog d2 = new Dog("Max");

d1.bark();  // Rex says Woof!
d2.bark();  // Max says Woof!
```

- Notice there is **only one copy of `bark()` code** in memory.
- But somehow, it uses **d1's name** in one call and **d2's name** in another.

Step 2: The Trick

- Every object keeps its **own copy of its data** (attributes) in memory.
  - `d1` has `name = "Rex"`
  - `d2` has `name = "Max"`
- The **method** doesn't store the data itself. Instead, it knows **which object is calling it**.

Step 3: The Magic Word: `this`

- In most OOP languages, a special hidden reference exists called **`this`** (Python: `self`).
- `this` **points to the current object** that called the method.

```
class Dog {
    String name;
    public Dog(String name) {
        this.name = name;
    }

    void bark() {
        System.out.println(this.name + " says Woof!");
    }
}
```

The Hidden Parameter

Definition:
যখন আমরা কোনো অবজেক্টের মেথড কল করি, কম্পাইলার গোপনভাবে সেই অবজেক্টের **মেমোরি ঠিকানা পাঠায়** মেথডে। এই লুকানো প্যারামিটারকে `this` (Java/C++) বা `self` (Python) বলা হয়।

Example (Java/Python):

```
Dog d1 = new Dog("Rex");
d1.bark();      // What we write
// Actually executed by compiler:
Dog_bark(d1);   // Hidden parameter passed
```

this (C++/Java) and self (Python)

Inside the method, we have access to this hidden parameter

Python makes this explicit. You MUST write self as the first parameter.

C++/Java makes this implicit. The keyword this is always available in the background.

Python

```
def bark(self):
        print(self.name + " says Woof!")
        # self is d1
```

Java

```
public void bark()
{
      System.out.println(this.name + " says Woof!"); // 'this' is implied,
      but you can write it.
}
```

Using this to resolve ambiguity:

The most common use of this in Java/C++ is when parameter names clash with attribute names.

```
class Student{
      private String name; // Attribute

      public Student(String name){ // Parameter // name = name; // Bug!
      Assigns parameter to itself.

      // Fix: Use 'this' to point to the Object's attribute.
      this.name = name;
   }
}
```

Defining the Rules (Invariants):

Before coding, define the rules your Class must protect:

Rule1: Awallet cannot have a negative balance.

Rule2: Youcannot send more money than you have.

Rule3: The owner's name is set at birth (constructor) and cannot be changed later.

These rules dictate our Encapsulation strategy. 'balance' must be private. 'owner' must have no setter method.

baki problem er solution gulo shhet theke dekhe nebo.